

AI Assisted Coding

Assignment 14.3

Name: R.Bharadwaj

Hall ticket no: 2303A51610

Batch no: 19

Task 1: AI-Assisted Personal Portfolio Website

Prompt:

Generate a clean semantic HTML5 portfolio website structure with sections:

Home, About, Projects, Contact.

Use proper tags like header, nav, section, footer.

Include a navigation bar with anchor links.

Add placeholder content for each section.

Generate responsive CSS using Flexbox for the portfolio layout.

Make navigation bar sticky.

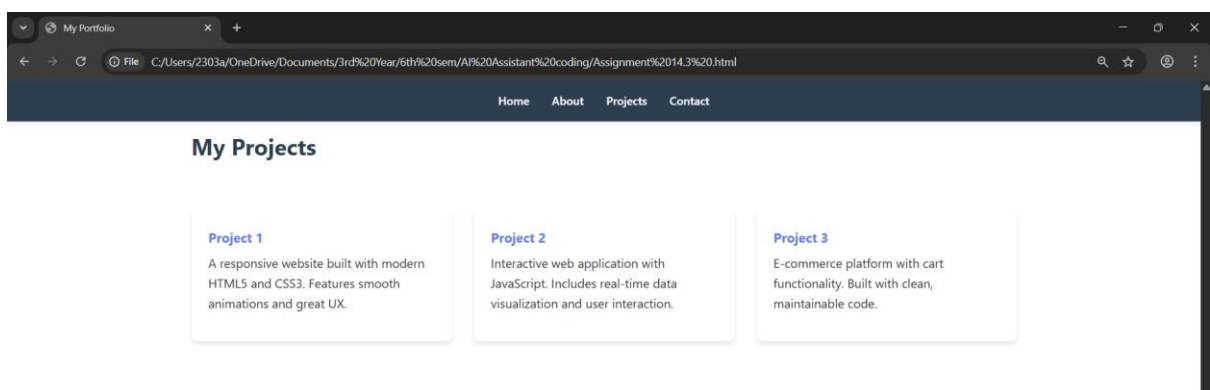
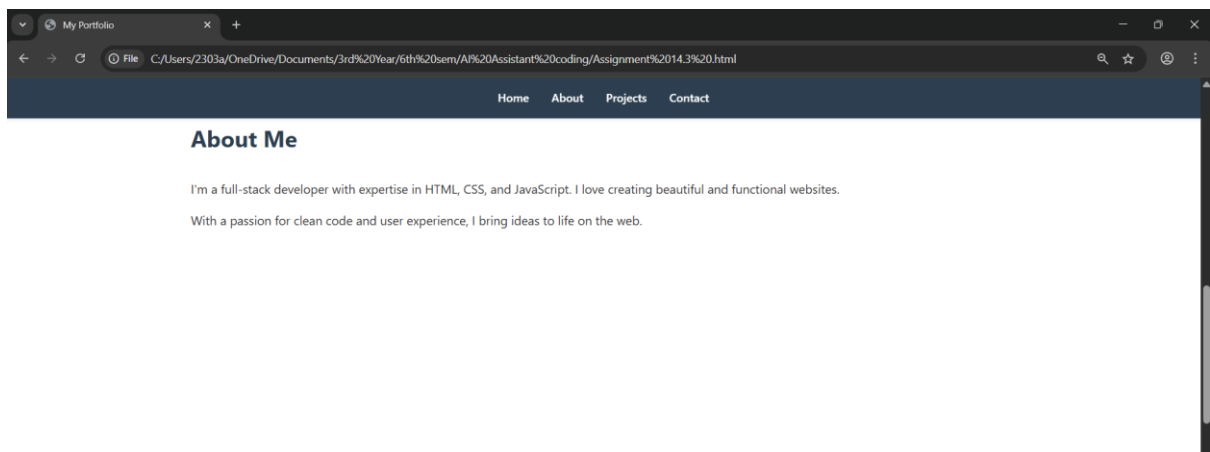
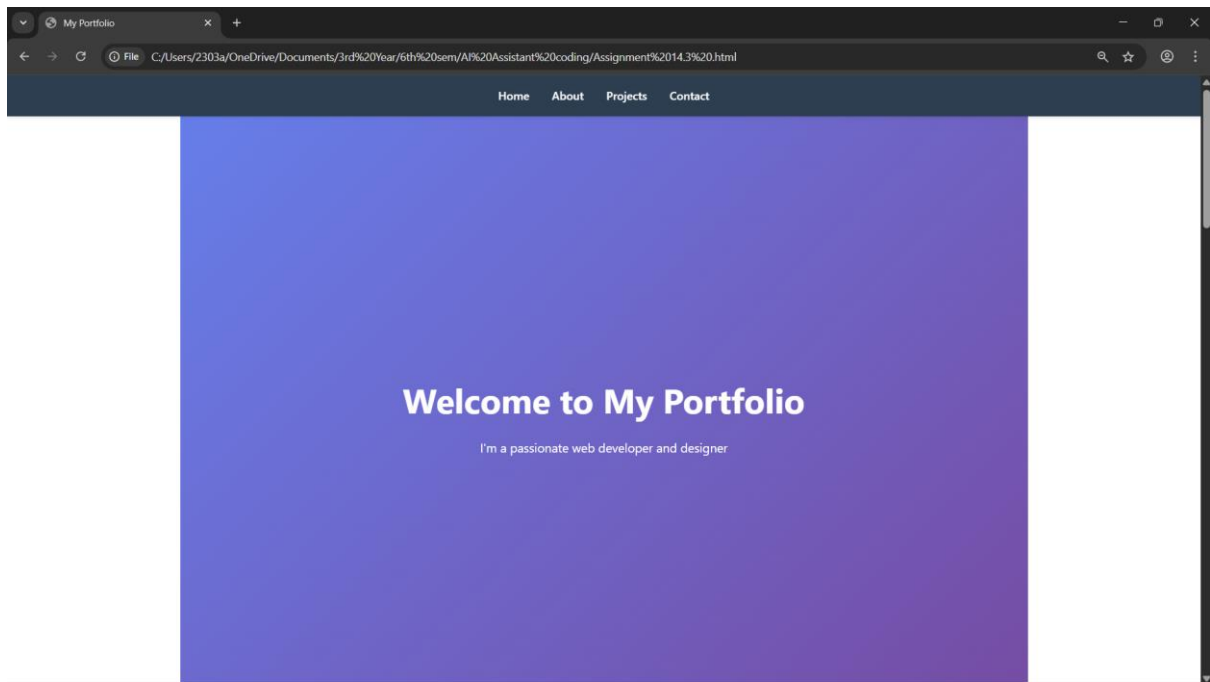
Style project cards in a grid layout.

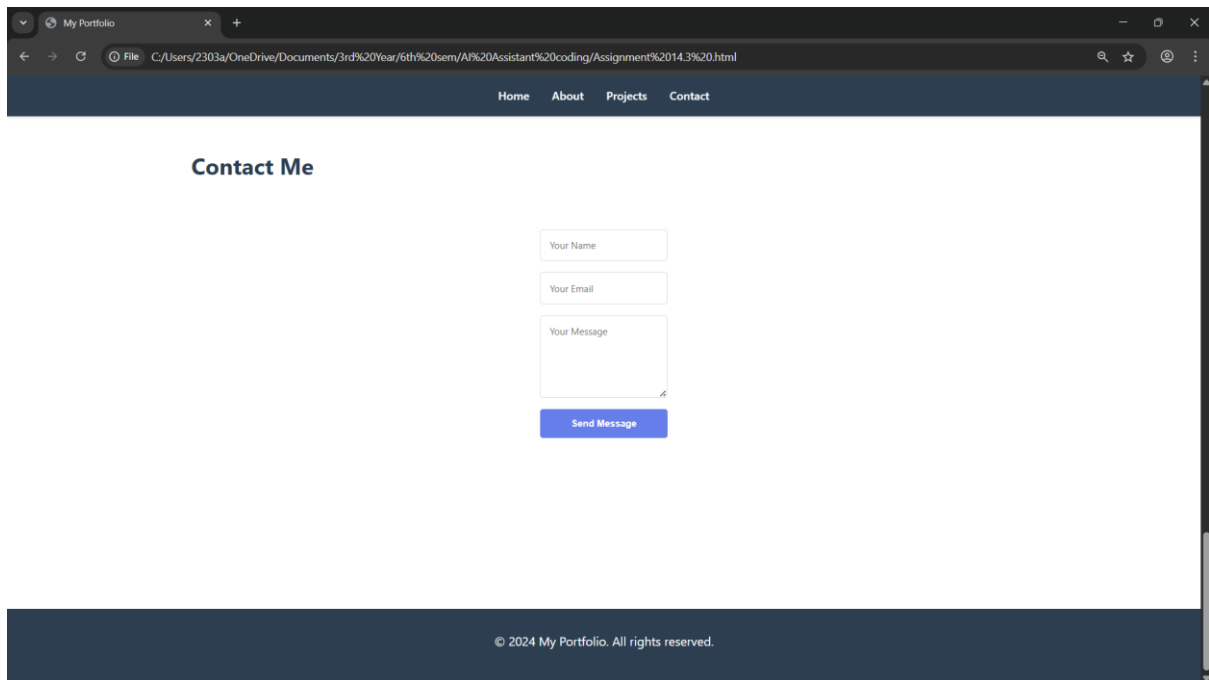
Add hover effects on project cards.

Write JavaScript to implement smooth scrolling when clicking navigation links.

Code & Output:

```
Assignment 14.3 .html Assignment 14.3 .html\html\head\style\section
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Portfolio - My Work</title>
7   <style>
8     * {
9       margin: 0;
10      padding: 0;
11      box-sizing: border-box;
12    }
13    body {
14      font-family: Arial, sans-serif;
15      line-height: 1.6;
16      color: #333;
17    }
18    header {
19      background-color: #2c3e50;
20      color: white;
21      padding: 1rem 0;
22      position: sticky;
23      top: 0;
24    }
25    nav a {
26      color: white;
27      text-decoration: none;
28      margin: 0 1.5rem;
29      transition: color 0.3s;
30    }
31    nav a:hover {
32      color: #3498db;
33    }
34    section {
35      padding: 3rem 2rem;
36      max-width: 1200px;
37      margin: 0 auto;
38    }
39    section h2 {
40      margin-bottom: 1.5rem;
41      color: #2c3e50;
```





Explanation:

The first task involves designing a personal portfolio website using HTML, CSS, and JavaScript with the help of AI tools like GitHub Copilot. The main objective of this task is to create a well-structured, responsive, and interactive web application that showcases a student's skills, projects, and contact information. Initially, a semantic HTML structure is generated, which includes sections such as Home, About, Projects, and Contact. Semantic tags like `<header>`, `<nav>`, `<section>`, and `<footer>` are used instead of generic `<div>` elements to improve readability, maintainability, and accessibility of the webpage. Navigation links are connected to different sections using anchor tags, allowing users to easily move across the website.

For styling, CSS is applied using modern layout techniques such as Flexbox and Grid to ensure the website is responsive across different devices like desktops, tablets, and mobile phones. Flexbox is mainly used for aligning elements such as the navigation bar, while Grid is used to organize project cards in a structured layout. Media queries are implemented to adjust the layout based on screen size, ensuring a consistent user experience. The AI-generated CSS is further customized by adding hover effects, smooth transitions, improved spacing, and color schemes to enhance the visual appeal. For example, project cards are designed to slightly scale up and display shadows when hovered, giving a more interactive and professional look.

JavaScript is used to improve the interactivity of the website, particularly by implementing smooth scrolling functionality. Instead of jumping abruptly to sections when navigation links are clicked, the page scrolls smoothly, providing a better user experience. This is achieved using methods like `scrollIntoView` with smooth behavior. Overall, this task helps in understanding how to effectively use AI tools for generating code while also emphasizing the importance of modifying and improving the generated output. It demonstrates the integration of structure (HTML), design (CSS), and behavior (JavaScript) to build a complete and user-friendly web application.

Task 2: AI-Generated Restaurant Landing Page

Prompt:

Create a restaurant landing page with a responsive navigation bar.

Include sections: Home, Menu, Gallery, Contact.

Use modern UI design with hero section.

Generate a responsive menu section using CSS cards.

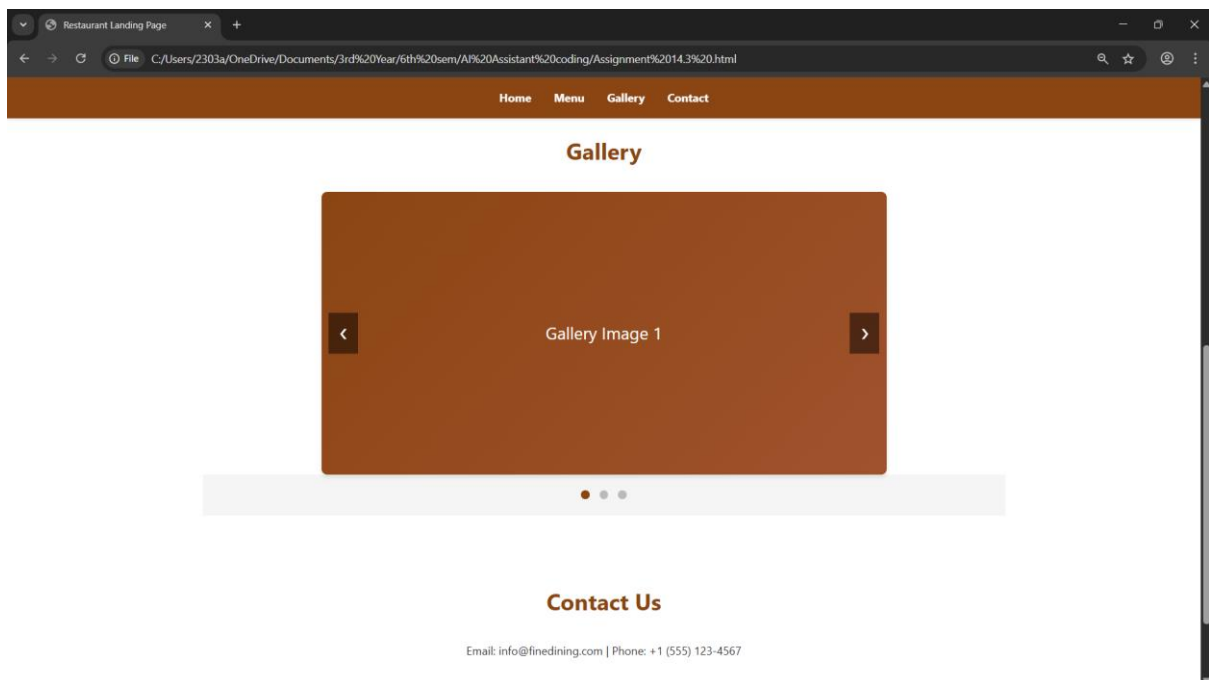
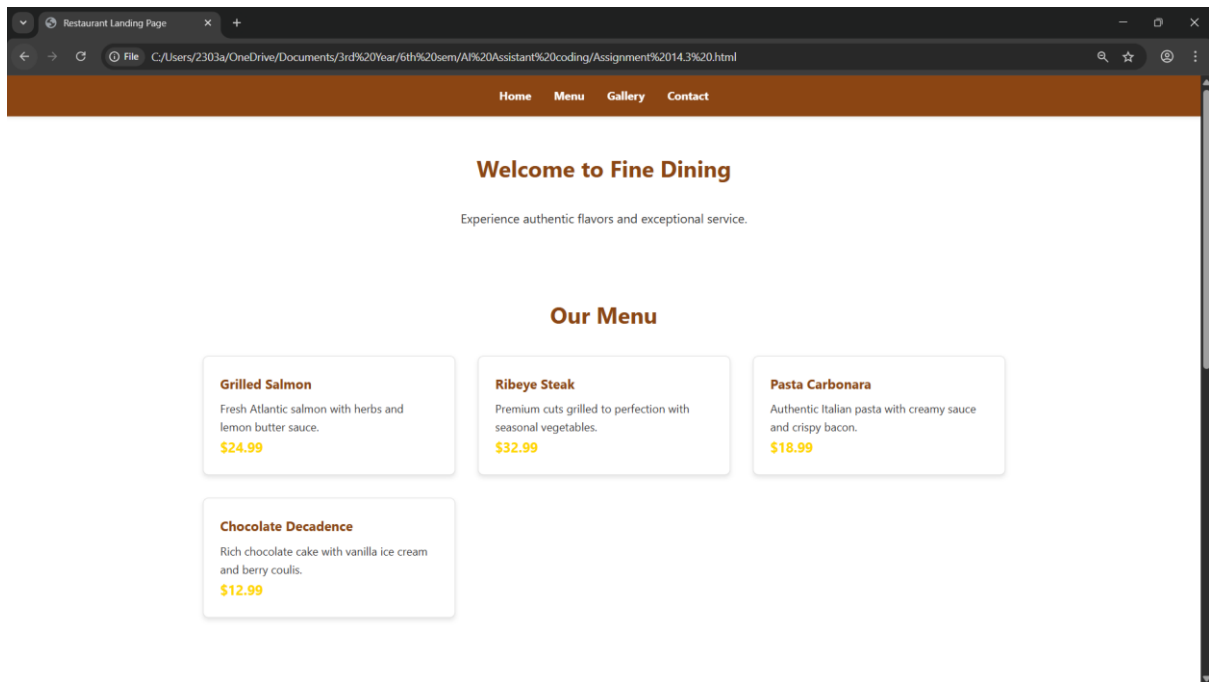
Each card should include image, dish name, price, and description.

Create a JavaScript image slider for a gallery section.

Include next and previous buttons with smooth transitions.

Code & Output:

```
Assignment 14.3 .html Assignment 14.3 .html\html\head\style
243 <!--
244 Task 2: AI-Generated Restaurant Landing Page
245
246 Create a restaurant landing page with a responsive navigation bar.
247 Include sections: Home, Menu, Gallery, Contact.
248 Use modern UI design with hero section.
249 Generate a responsive menu section using CSS cards.
250 Each card should include image, dish name, price, and description.
251 Create a JavaScript image slider for a gallery section.
252 Include next and previous buttons with smooth transitions.
253 -->
254 <!DOCTYPE html>
255 <html lang="en">
256 <head>
257   <meta charset="UTF-8">
258   <meta name="viewport" content="width=device-width, initial-scale=1.0">
259   <title>Restaurant Landing Page</title>
260   <style>
261     * {
262       margin: 0;
263       padding: 0;
264       box-sizing: border-box;
265     }
266
267     body {
268       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
269       line-height: 1.6;
270       color: #333;
271     }
272
273     /* Navigation */
274     header {
275       background: #1a1a1a;
276       color: white;
277       padding: 1rem 0;
278       position: sticky;
279       top: 0;
280       z-index: 100;
281       box-shadow: 0 2px 5px rgba(0,0,0,0.2);
282     }
283
```



Explanation:

The second task focuses on developing a restaurant landing page using HTML, CSS, and JavaScript with the assistance of AI tools like GitHub Copilot. The objective of this task is to create a visually appealing and user-friendly interface that effectively showcases the restaurant's offerings. Initially, a responsive navigation bar is generated using HTML and CSS, which includes links such as Home, Menu, Gallery, and Contact. This navigation bar allows users to easily access different sections of the webpage and is designed to adapt to various screen sizes for better usability on both desktop and mobile devices.

The menu section is implemented using CSS card layouts, where each card represents a food item with an image, name, price, and short description. Modern CSS techniques like Flexbox or Grid are

used to organize these cards in a structured and responsive manner. The AI-generated design is further improved by customizing styles such as spacing, colors, and hover effects to enhance visual appeal and user interaction. For instance, when a user hovers over a menu card, it slightly enlarges or changes appearance, making the interface more dynamic and engaging.

Additionally, a JavaScript-based image slider is implemented for the gallery section to display multiple images of the restaurant or dishes. The slider includes navigation controls like next and previous buttons and uses smooth transitions to create a seamless viewing experience. Further enhancements such as automatic sliding and fade effects can be added to improve functionality and user experience. Overall, this task demonstrates how AI tools can accelerate front-end development while still requiring manual customization and logical improvements. It highlights the integration of structured layout, responsive design, and interactive features to build a complete and attractive landing page.

Task 3: AI-Powered Event Registration Form

Prompt:

Generate an HTML form for event registration with fields:

Name, Email, Phone Number, Department dropdown, Event selection dropdown.

Include submit button.

Style the form to be clean, centered, and user-friendly.

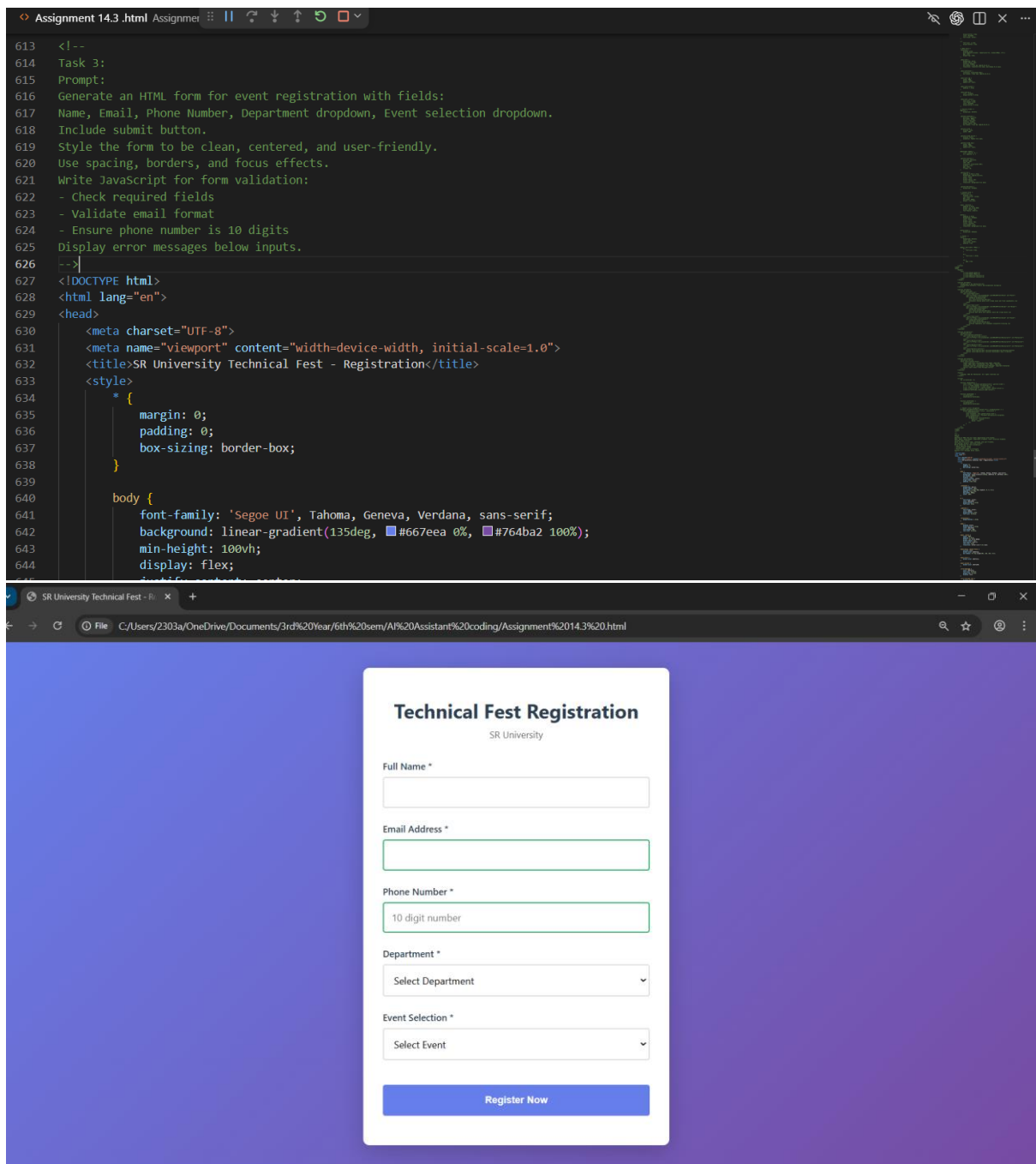
Use spacing, borders, and focus effects.

Write JavaScript for form validation:

- Check required fields
- Validate email format
- Ensure phone number is 10 digits

Display error messages below inputs.

Code & Output:



Explanation:

The third task involves developing an AI-powered event registration form using HTML, CSS, and JavaScript with the assistance of GitHub Copilot. The main objective of this task is to create a user-friendly and responsive form that collects accurate input from users while ensuring proper validation. Initially, the HTML structure of the form is generated, which includes input fields such as Name, Email, Phone Number, Department, and Event Selection, along with a submit button. These fields are organized in a structured manner to maintain clarity and ease of use for the user.

CSS is then applied to enhance the visual appearance and usability of the form. The form is styled to be clean, centered, and accessible, with proper spacing, borders, and alignment. Focus effects are added to input fields to highlight the active field, improving the overall user experience. The AI-

generated CSS is further customized to make the design more visually appealing by adjusting colors, adding padding, and ensuring responsiveness across different screen sizes.

JavaScript plays a crucial role in this task by implementing real-time validation for user inputs. Validation checks include ensuring that all required fields are filled, verifying the correct format of the email address, and confirming that the phone number contains exactly 10 digits. Instead of allowing incorrect data submission, error messages are displayed near the respective input fields, guiding the user to correct their input. Additional improvements such as changing border colors based on input validity and preventing form submission until all validations are satisfied enhance both functionality and user experience. Overall, this task demonstrates how AI-assisted development can be combined with logical validation techniques to create an efficient, interactive, and reliable web form.

Task 4: AI-Assisted E-Commerce Product Page

Prompt:

Create a responsive product catalog using CSS Grid.

Each product should have image, name, price, and Add to Cart button.

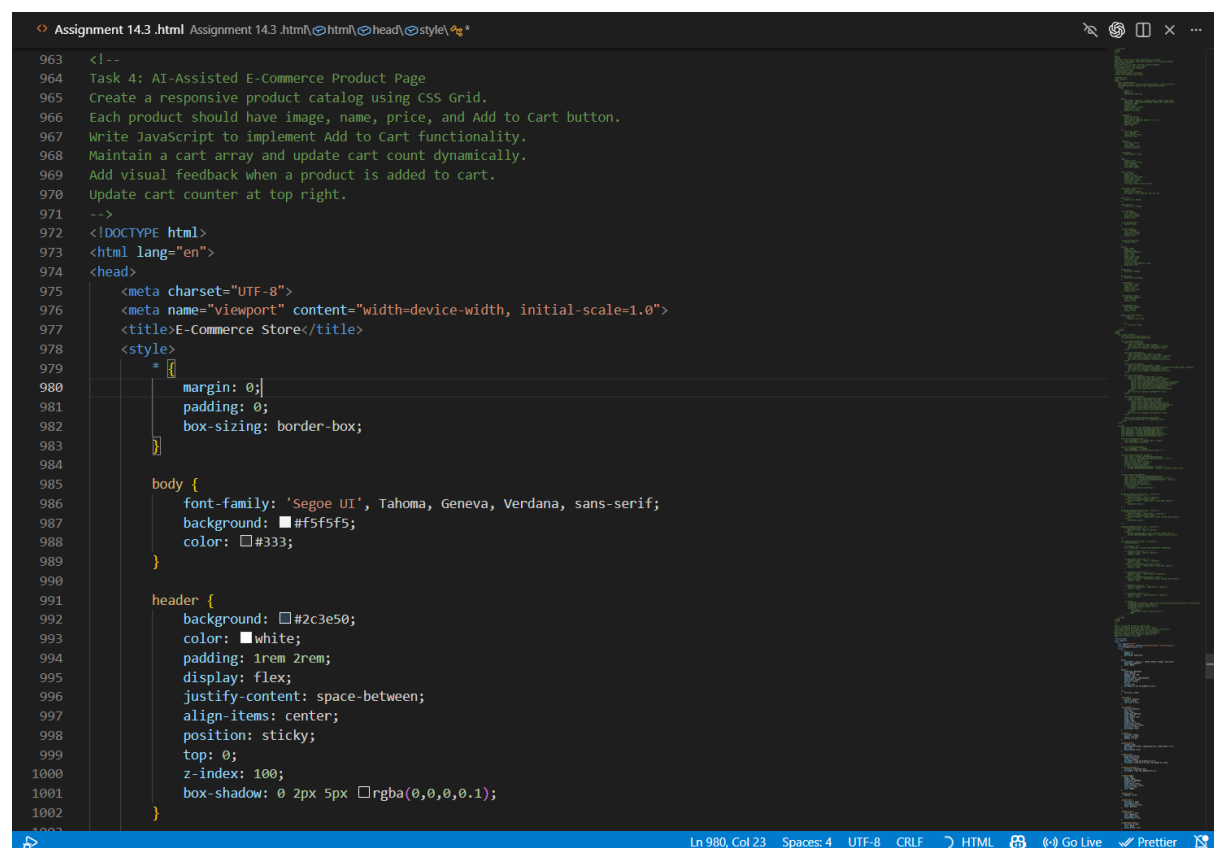
Write JavaScript to implement Add to Cart functionality.

Maintain a cart array and update cart count dynamically.

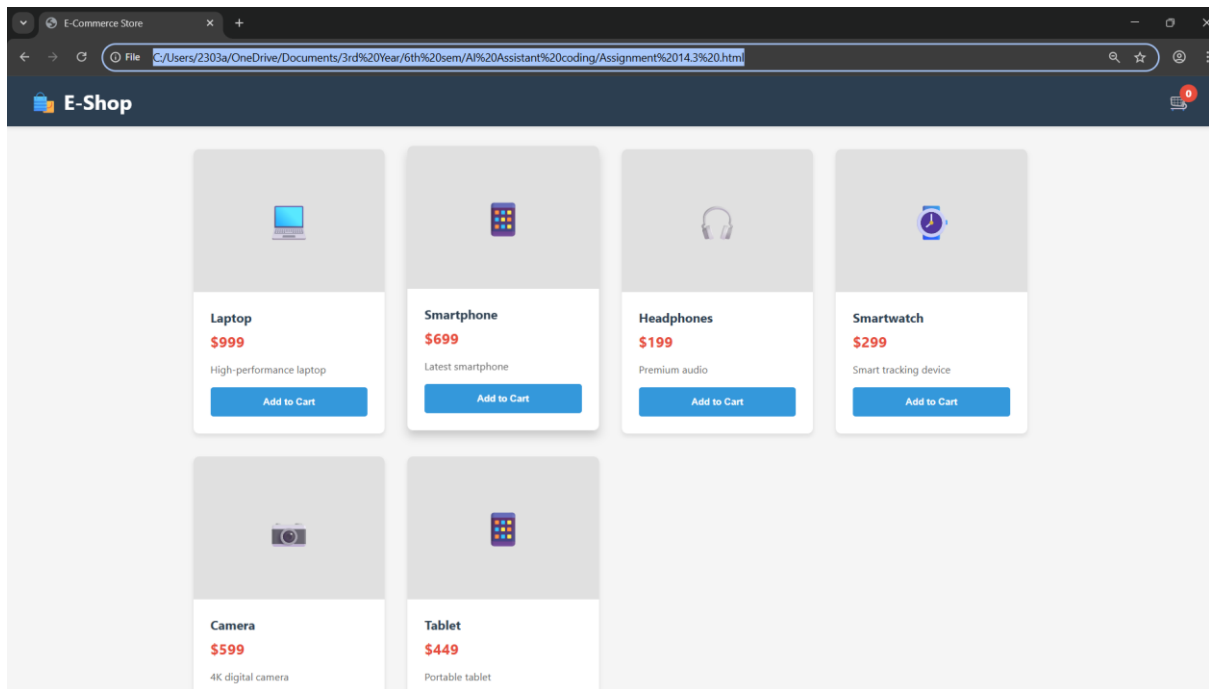
Add visual feedback when a product is added to cart.

Update cart counter at top right.

Code & Output:



```
Assignment 14.3 .html Assignment 14.3 .html\html\head\style\*
963 <!--
964 Task 4: AI-Assisted E-Commerce Product Page
965 Create a responsive product catalog using CSS Grid.
966 Each product should have image, name, price, and Add to Cart button.
967 Write JavaScript to implement Add to Cart functionality.
968 Maintain a cart array and update cart count dynamically.
969 Add visual feedback when a product is added to cart.
970 Update cart counter at top right.
971 -->
972 <!DOCTYPE html>
973 <html lang="en">
974 <head>
975   <meta charset="UTF-8">
976   <meta name="viewport" content="width=device-width, initial-scale=1.0">
977   <title>E-Commerce Store</title>
978   <style>
979     * {
980       margin: 0;
981       padding: 0;
982       box-sizing: border-box;
983     }
984
985     body {
986       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
987       background-color: #f5f5f5;
988       color: #333;
989     }
990
991     header {
992       background-color: #2c3e50;
993       color: white;
994       padding: 1rem 2rem;
995       display: flex;
996       justify-content: space-between;
997       align-items: center;
998       position: sticky;
999       top: 0;
1000       z-index: 100;
1001       box-shadow: 0 2px 5px rgba(0,0,0,0.1);
1002     }
1003   </style>
```

Explanation:

The fourth task involves developing a basic e-commerce product page using HTML, CSS, and JavaScript with the assistance of AI tools like GitHub Copilot. The objective of this task is to create a functional and interactive interface where users can view products and perform simple cart operations. Initially, a product catalog is generated using HTML and styled with CSS Grid to display multiple products in a structured layout. Each product card includes elements such as an image, product name, price, and an “Add to Cart” button. CSS Grid is used to ensure that the layout remains responsive, automatically adjusting the number of columns based on the screen size for better viewing on different devices.

The styling is further enhanced by customizing the AI-generated CSS to improve visual appeal and user interaction. Spacing, alignment, and color schemes are refined, and hover effects are added to product cards to make the interface more engaging. For example, when a user hovers over a product card, it may slightly enlarge or display a shadow effect, giving a more modern and interactive feel. These modifications ensure that the design does not look generic and improves the overall user experience.

JavaScript is used to implement the core functionality of the “Add to Cart” feature. When a user clicks the button, the selected product is added to a cart data structure, and the cart item count is updated dynamically, usually displayed at the top-right corner of the page. Additional enhancements such as providing visual feedback when a product is added, like temporarily changing the button text to “Added” or highlighting the cart icon, improve usability. Basic checks can also be included to prevent duplicate additions or excessive clicking. Overall, this task demonstrates the integration of structured layout, responsive design, and dynamic functionality, while also emphasizing the importance of modifying AI-generated code to create a more refined and user-friendly e-commerce interface.

Task 5: Create a Responsive Web Page Layout

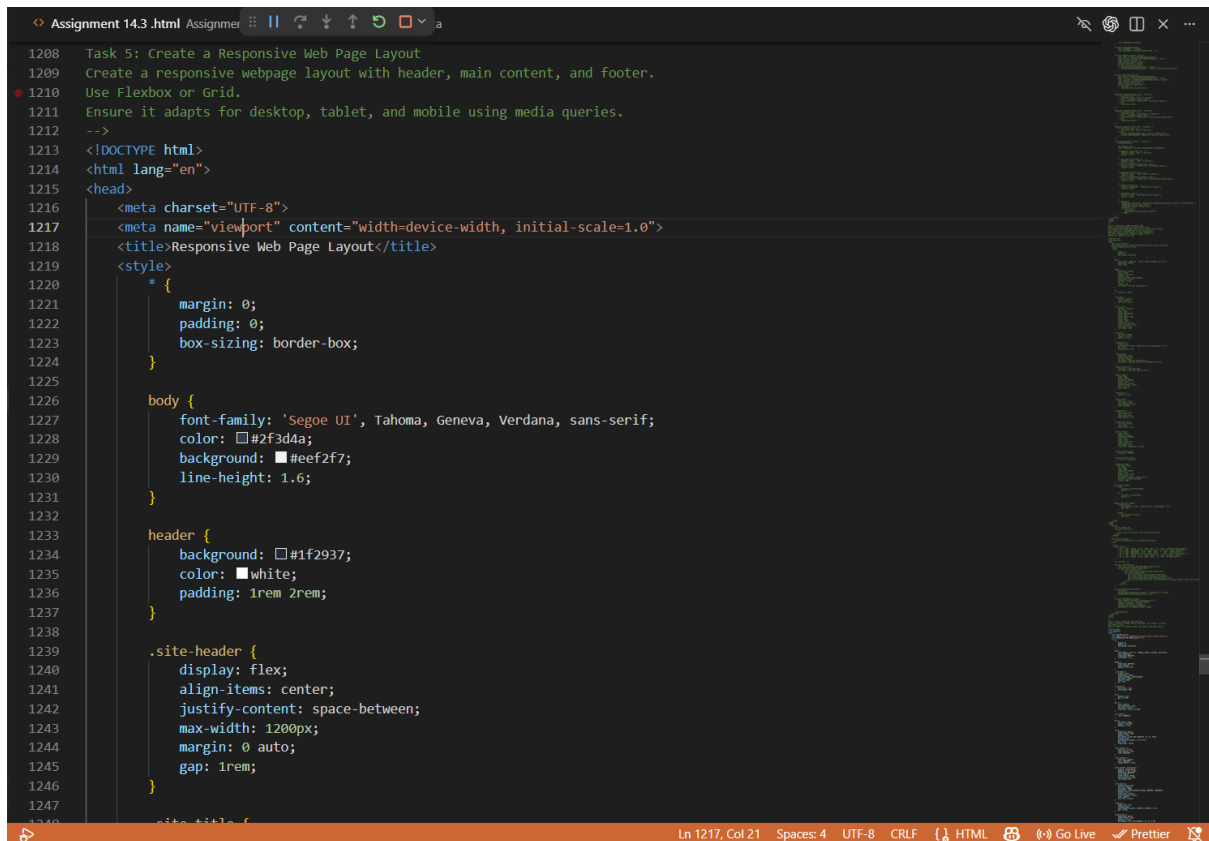
Prompt:

Create a responsive webpage layout with header, main content, and footer.

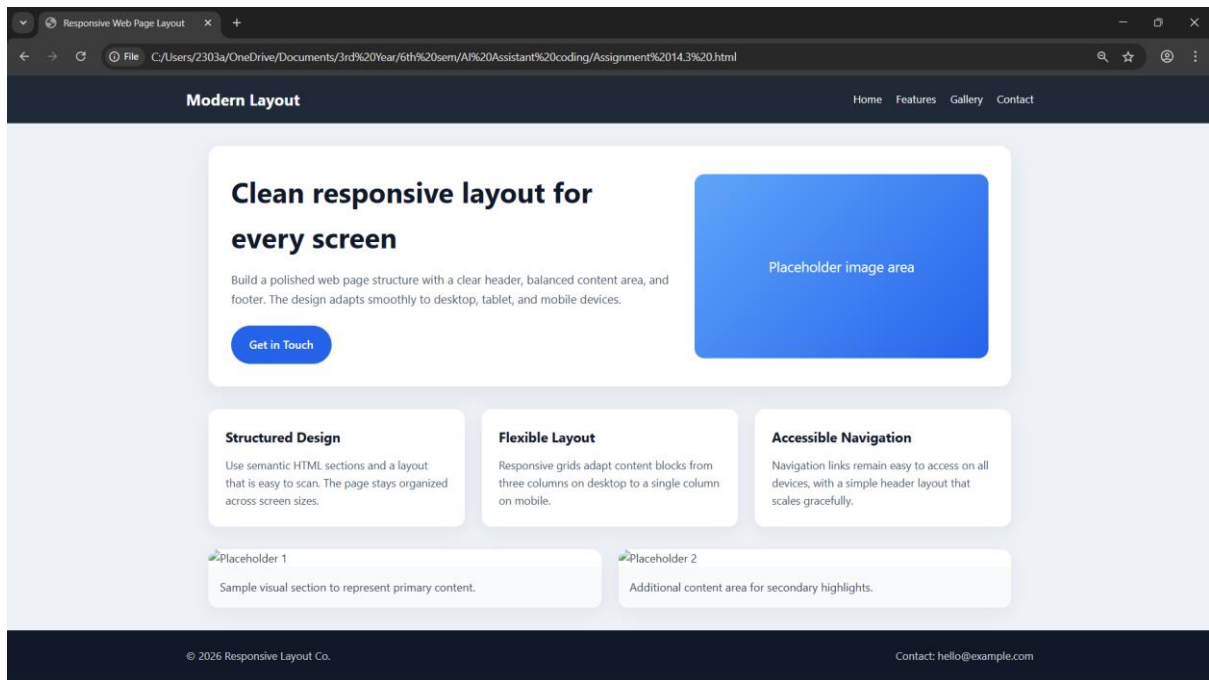
Use Flexbox or Grid.

Ensure it adapts for desktop, tablet, and mobile using media queries.

Code & Output:



```
1208 Task 5: Create a Responsive Web Page Layout
1209 Create a responsive webpage layout with header, main content, and footer.
1210 Use Flexbox or Grid.
1211 Ensure it adapts for desktop, tablet, and mobile using media queries.
1212 -->
1213 <!DOCTYPE html>
1214 <html lang="en">
1215 <head>
1216   <meta charset="UTF-8">
1217   <meta name="viewport" content="width=device-width, initial-scale=1.0">
1218   <title>Responsive Web Page Layout</title>
1219   <style>
1220     * {
1221       margin: 0;
1222       padding: 0;
1223       box-sizing: border-box;
1224     }
1225
1226     body {
1227       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
1228       color: #2f3d4a;
1229       background: #eef2f7;
1230       line-height: 1.6;
1231     }
1232
1233     header {
1234       background: #1f2937;
1235       color: white;
1236       padding: 1rem 2rem;
1237     }
1238
1239     .site-header {
1240       display: flex;
1241       align-items: center;
1242       justify-content: space-between;
1243       max-width: 1200px;
1244       margin: 0 auto;
1245       gap: 1rem;
1246     }
1247
1248     .site-title {
```



Explanation:

The fifth task involves creating a responsive web page layout using HTML and CSS with the help of AI tools like GitHub Copilot. The main objective of this task is to design a clean and well-organized webpage structure that adapts effectively to different screen sizes such as desktops, tablets, and mobile devices. Initially, a basic HTML layout is generated, which includes three main sections: a header, a main content area, and a footer. The header typically contains navigation links, the main section includes placeholder text or images to represent content, and the footer provides additional information such as copyright or contact details.

CSS is then used to style and structure the layout using modern techniques like Flexbox or Grid. Flexbox is commonly used to align elements within the header and footer, while Grid or Flexbox can be used to organize the main content area. The layout is designed to be visually balanced with proper spacing, alignment, and consistent styling. To ensure responsiveness, media queries are implemented to adjust the layout based on screen width. For example, elements that appear side-by-side on larger screens may stack vertically on smaller devices, improving readability and usability.

The AI-generated CSS is further refined by customizing breakpoints, improving spacing, and enhancing visual consistency. Adjustments such as resizing fonts, modifying padding, and reorganizing content for smaller screens ensure that the webpage remains user-friendly across all devices. This task highlights the importance of responsive design in modern web development and demonstrates how AI tools can assist in generating layouts while still requiring manual improvements to achieve a polished and professional result.